

Sentencias básicas de SQL – ejemplos de Impocheck

1. INSERT

Agrega nuevos registros.

- **Ejemplo:** INSERT INTO Areas (id_area, descripcion) VALUES (6, 'Logística');

```
INSERT INTO Personas (DNI, nombre, apellido, fecha_nacimiento, domicilio)
VALUES
(32000100, 'Carlos', 'Gómez', '1985-05-20', 'Av. Rivadavia 1234'),
(34000200, 'Ana', 'Martínez', '1988-11-12', 'Calle Corrientes 567');

-- 2. Los damos de alta como Empleados (Referencia por DNI)
-- Usamos CUIL de 12 dígitos y Legajos únicos
INSERT INTO Empleados (DNI, fecha_ingreso, CUIL, num_legajo, id_area)
VALUES
(32000100, '2024-01-15', '203200010001', 5001, 1), -- Administración
(34000200, '2023-06-10', '273400020002', 5002, 3) -- Ventas

-- 3. Les asignamos sus Hijos (DNI de hijos entre 6 y 9 dígitos)
INSERT INTO Hijo_Empleado (DNI_Empleado, DNI_Hijo)
VALUES
(32000100, 55000111), -- Hijo de Carlos
(34000200, 58000222); -- Hijo de Ana
```

```
INSERT INTO Distribuidores (
    CUIT, RazonSocial, NombreFantasia, DireccionFiscal, email,
    ZonaCobertura, limiteCredito, condicionFiscal, fechaAlta
)
VALUES (
    '30999888770',
    'Loginter',
    'Centro Distribución Especializado',
    'Calle 50 nro 123, La Plata',
    'logistica@centro.com.ar',
    'GBA Sur',
    1500000.00,
    'Responsable Inscripto',
    GETDATE()
);

-- 2. Insertamos sus 2 direcciones de entrega vinculadas al CUIT
INSERT INTO DireccionesDistribuidores (CUIT, codigo_direccion, direccion, telefono)
VALUES
('30999888770', 1, 'Depósito Avellaneda - Calle 1 nro 500', 42001111),
('30999888770', 2, 'Punto Retiro Quilmes - Av. Mitre 900', 42002222);
```

```

--nuevo proveedor
DECLARE @NuevoProvID INT;

-- 1. Insertamos el Proveedor
INSERT INTO Proveedores (tax_id, nombre_empresa, pais, sitio_web, id_moneda, banco, CBU)
VALUES (
    'TAX-2026-X',
    'Suministros Globales S.A.',
    'Argentina',
    '://suministrosglobales.com',
    1, -- Referencia a Pesos
    'Banco Francés',
    '0170001234567' -- CBU de 13 dígitos
);

-- Capturamos el ID recién creado
SET @NuevoProvID = SCOPE_IDENTITY();

-- 2. Insertamos los 2 mails asociados
INSERT INTO Proveedores_Mails (Id_interno_Prov, mail, tipo_contacto)
VALUES
(@NuevoProvID, 'ventas@suministros.com', 'Comercial'),
(@NuevoProvID, 'administracion@suministros.com', 'Facturación');

```

```

-- Insertamos el nuevo artículo
INSERT INTO articulos (NumeroArticulo, Nombre, Descripcion, UnidadMedida,
VolumenM3, UnidadesPorBulto, PesoKilos, Color, CodigoEAN13, fechaIngreso)
VALUES (5002, 'Monitor Led 35', 'Monitor Full HD 35 pulgadas ', 'Unidad',
0.0500, 1, 3.50, 'Gris', '77900000000051', GETDATE());
-- Capturamos el ID generado y le asignamos precio en la Lista 1
DECLARE @NuevoID INT = SCOPE_IDENTITY();
INSERT INTO articulos_Precios (Cod_Listas_Precios, ArtículoID, precio)
VALUES (1, @NuevoID, 160000.00);

```

```

-- insertar nueva condicion de pago
declare @nuevaCP int = (select (max(cod_condicion_pago))+1 from Condiciones_Pago)
insert into Condiciones_Pago
values (@nuevaCP, '90/ 120 días', 2)

```

*VARIABLES:

- Declaracion:
 - **DECLARE** @(nombre) (tipo de dato)
- Asignacion de valor:
 - **SET** @(nombre) = (valor)
 - **DECLARE** @(nombre) (tipo de dato) = (valor)

Los nombres de las variables siempre van precedidos por "@", y los valores y el tipo de dato elegido deben ser compatibles entre si

2. DELETE

Borra registros.

- **Ejemplo:** DELETE FROM Licencias_empleados WHERE id_licencia = 1;

3. UPDATE

Modifica registros existentes.

- **Ejemplo:** UPDATE articulos SET Color = 'Gris' WHERE Color = 'Negro';

```
UPDATE Distribuidores
SET limiteCredito = limiteCredito * 1.20
WHERE ZonaCobertura = 'Zona Norte';
```

4. SELECT

Selecciona las columnas que querés ver.

- **Ejemplo:** SELECT Nombre, Color FROM articulos;

5. WHERE

Filtra filas según una condición.

- **Ejemplo:** SELECT * FROM Personas WHERE DNI = 20000001;

6. ORDER BY

Ordena los resultados (ASC por defecto o DESC).

- **Ejemplo:** SELECT * FROM articulos ORDER BY PesoKilos DESC;

7. WHERE + AND / OR

Combina múltiples condiciones de filtrado.

- **Ejemplo:** SELECT * FROM articulos WHERE Color = 'Rojo' AND PesoKilos > 2;

8. LIKE

Busca patrones de texto usando %.

- **Ejemplo:** SELECT * FROM Distribuidores WHERE RazonSocial LIKE 'Empresa%';

9. DISTINCT

Elimina filas duplicadas del resultado.

- **Ejemplo:** SELECT DISTINCT Pais FROM Proveedores;

10. TOP

Limita la cantidad de filas devueltas (TOP en SQL Server).

- **Ejemplo:** SELECT TOP 5 * FROM Compras ORDER BY fecha_alta DESC;

11. AGGREGATES (SUM, AVG, COUNT, etc.)

Realiza cálculos sobre un conjunto de valores.

- **Ejemplo:** SELECT COUNT(*) AS TotalEmpleados FROM Empleados;

```
SELECT COMPRAS.Numero_OC, COMPRAS.Id_interno_Prov, SUM(Items_Compra.cantidad) AS TotalComprado
, MAX(Items_Compra.cantidad) as MaxComprado ,MIN (Items_Compra.cantidad) as MinComprado
FROM COMPRAS
INNER JOIN Items_Compra ON COMPRAS.Numero_OC = Items_Compra.Numero_OC
GROUP BY COMPRAS.Numero_OC, COMPRAS.Id_interno_Prov
```

12. GROUP BY / HAVING

Agrupar filas y filtra esos grupos (HAVING es el WHERE de los grupos).

- **Ejemplo:** SELECT id_area, COUNT(*) FROM Empleados GROUP BY id_area HAVING COUNT(*) > 2;

```
SELECT Empleados.id_area, areas.descripcion, COUNT(*)
FROM Empleados inner join Areas
on empleados.id_area = areas.id_area
GROUP BY Empleados.id_area, areas.descripcion
HAVING COUNT(*) > 2;
```

13. INNER JOIN

Devuelve filas cuando hay coincidencia en ambas tablas.

- **Ejemplo:** SELECT E.num_legajo, A.descripcion FROM Empleados E INNER JOIN Areas A ON E.id_area = A.id_area;

```
SELECT
    P.nombre,
    P.apellido,
    E.num_legajo,
    A.descripcion AS Nombre_Area
FROM Personas P
INNER JOIN Empleados E ON P.DNI = E.DNI -- Unimos Persona con su rol de Empleado
INNER JOIN Areas A ON E.id_area = A.id_area; -- Unimos Empleado con su Área
```

14. LEFT JOIN

Devuelve todas las filas de la tabla izquierda, aunque no tengan coincidencia en la derecha.

- **Ejemplo:** SELECT P.nombre, H.DNI_Hijo FROM Empleados P LEFT JOIN Hijo_Empleado H ON P.DNI = H.DNI_Empleado;

```
SELECT P.DNI , H.DNI_Hijo
FROM Empleados P
LEFT JOIN Hijo_Empleado H
ON P.DNI = H.DNI_Empleado
-- devuelve todos los empleados aunque NO
--tengan hijos
```

15. RIGHT JOIN

Igual que el Left, pero prioriza la tabla de la derecha.

- **Ejemplo:** SELECT M.mail, P.nombre_empresa FROM Proveedores_Mails M RIGHT JOIN Proveedores P ON M.Id_interno_Prov = P.Id_interno_Prov;
- Este es un ejemplo de Jurasik park que vimos en clase

```
SELECT Apellido_Guia as Apellido, Nombre_Guia as Nombre,
rtv.Codigo_Tipo_Visita as 'Tipo Visita'
FROM reserva_tipo_visita rtv
right join Guia
on rtv.Codigo_Guia = guia.Codigo_Guia
WHERE rtv.Codigo_Tipo_Visita is null
-- Guías NO asignados aun a ninguna visita
```

16. FULL JOIN

Devuelve todos los registros cuando hay coincidencia en una de las tablas (izquierda o derecha).

- **Ejemplo:** SELECT * FROM Compras C FULL JOIN Compras_Aprobadas CA ON C.Numero_OC = CA.Numero_OC;

17. UNION

Combina resultados de dos consultas (elimina duplicados).

- **Ejemplo:** SELECT CUIT FROM Distribuidores UNION SELECT tax_id FROM Proveedores;

```
SELECT CUIT, RazonSocial
FROM Distribuidores
UNION
SELECT tax_id, nombre_empresa
FROM Proveedores
```

18. INTERSECT

Devuelve solo los registros que aparecen en ambas consultas.

- **Ejemplo:** SELECT DNI FROM Personas INTERSECT SELECT DNI FROM Empleados;

```
SELECT DNI
FROM Personas
INTERSECT
SELECT DNI
FROM Empleados
--(personas que SI son empleados)

Select Personas.DNI
FROM Personas INNER JOIN
Empleados ON Personas.DNI = Empleados.DNI
-- PERSONAS QUE SI SON EMPLEADOS
```

19. EXCEPT

Devuelve registros de la primera consulta que NO están en la segunda.

- **Ejemplo:** SELECT DNI FROM Personas EXCEPT SELECT DNI FROM Empleados; (Personas que no son empleados).

```

SELECT DNI
FROM Personas
EXCEPT
SELECT DNI
FROM Empleados
--(Personas que no son empleados).
select personas.DNI
from Personas
LEFT join Empleados on Personas.DNI = Empleados.DNI
where Empleados.DNI is null
--PERSONAS QUE NO SON EMPLEADOS

```

20. IN

Permite especificar múltiples valores en un WHERE.

- **Ejemplo:** SELECT * FROM Areas WHERE descripcion IN ('Ventas', 'Compras');

21. BETWEEN

Filtra valores dentro de un rango inclusivo.

- **Ejemplo:** SELECT * FROM Compras WHERE fecha_alta BETWEEN '2026-01-01' AND '2026-03-31';

22. IS NULL

Busca valores vacíos.

- **Ejemplo:** SELECT * FROM articulos WHERE FechaBaja IS NULL;

23. IS NOT NULL

Busca valores que no estén vacíos.

- **Ejemplo:** SELECT * FROM Proveedores WHERE sitio_web IS NOT NULL;

24. CASE

Lógica condicional (Si... Entonces...) dentro de una consulta.

- **Ejemplo:** SELECT Nombre, CASE WHEN PesoKilos > 10 THEN 'Pesado' ELSE 'Liviano' END FROM articulos;

25. SUBCONSULTAS

Una consulta dentro de otra. Pueden utilizarse en diferentes partes de las sentencias.

FROM: Cuando colocas una subconsulta dentro de la cláusula FROM, lo que estás haciendo es crear una **tabla temporal en memoria** (a menudo llamada *tabla derivada* o *vista en línea*). El motor de la base de datos ejecuta primero esa subconsulta y luego la consulta principal la utiliza como si fuera una tabla real.

Este enfoque es excelente para realizar **pre-agrupaciones o cálculos complejos** antes de mezclarlos con otras tablas.

- Ejemplo: --Ranking de empleados con más hijos. Queremos saber cuántos hijos tiene cada empleado, pero solo nos interesan aquellos que tienen más de un hijo, mostrando sus datos personales y el conteo.

```
SELECT
    E.num_legajo,
    P.apellido,
    P.nombre,
    ConteoHijos.Cantidad_Hijos
FROM Empleados E
INNER JOIN Personas P ON E.DNI = P.DNI
-- Subconsulta en el FROM que pre-calcula los hijos por empleado
INNER JOIN (
    SELECT DNI_Empleado, COUNT(*) AS Cantidad_Hijos
    FROM Hijo_Empleado
    GROUP BY DNI_Empleado
) AS ConteoHijos ON E.DNI = ConteoHijos.DNI_Empleado
WHERE ConteoHijos.Cantidad_Hijos > 1;
```

```
--Ranking de empleados con más hijos. Queremos saber cuántos hijos tiene cada empleado,
--pero solo nos interesan aquellos que tienen más de un hijo,
--mostrando sus datos personales y el conteo.
SELECT
    E.num_legajo,
    P.apellido,
    P.nombre,
    ConteoHijos.Cantidad_Hijos
FROM Empleados E
INNER JOIN Personas P ON E.DNI = P.DNI
-- Subconsulta en el FROM que pre-calcula los hijos por empleado
INNER JOIN (
    SELECT DNI_Empleado, COUNT(*) AS Cantidad_Hijos
    FROM Hijo_Empleado
    GROUP BY DNI_Empleado
) AS ConteoHijos ON E.DNI = ConteoHijos.DNI_Empleado
WHERE ConteoHijos.Cantidad_Hijos > 1;
```



```
--Auditoría de depósitos sobreocupados. Queremos listar los depósitos
--cuya sumatoria de capacidad de sus ubicaciones supera
--a la capacidad_total_m3 declarada en la cabecera del depósito (un error grave de carga de datos).

SELECT
    D.Cod_deposito,
    D.descripcion,
    D.capacidad_total_m3,
    SumaUbicaciones.Total_Ubicaciones_m3
FROM Depositos D
-- Subconsulta en el FROM que sumaliza los metros cúbicos de las ubicaciones
INNER JOIN (
    SELECT Cod_deposito, SUM(capacidad_ubicacion_m3) AS Total_Ubicaciones_m3
    FROM Ubicaciones
    GROUP BY Cod_deposito
) AS SumaUbicaciones ON D.Cod_deposito = SumaUbicaciones.Cod_deposito
WHERE SumaUbicaciones.Total_Ubicaciones_m3 > D.capacidad_total_m3;
```

- **El Alias es obligatorio:** Siempre, al cerrar el paréntesis de la subconsulta, debes asignarle un nombre (por ejemplo: AS ConteoHijos). Si lo omites, el motor SQL arrojará un error de sintaxis inmediatamente.

WHERE: Las subconsultas en la cláusula WHERE actúan como **filtros dinámicos**. En lugar de escribir un valor fijo (como WHERE precio > 500), dejas que una subconsulta calcule ese valor en tiempo de ejecución.

- A diferencia de las subconsultas en el FROM, estas suelen devolver **un único valor** (subconsultas escalares) o **una lista de una sola columna** (para usar con operadores como IN, NOT IN, >, etc.).
- Ejemplos:

```
--Encontrar los artículos más caros de la empresa. Queremos listar los datos de los artículos
--cuyo precio en la Lista de Precios General (Lista 1) sea igual al precio máximo registrado en todo el sistema.

SELECT ArticuloID, Nombre, Descripcion
FROM articulos
WHERE ArticuloID = (
    -- Esta subconsulta devuelve un único número (el ID del artículo más caro)
    SELECT TOP 1 ArticuloID
    FROM articulos_Precios
    WHERE Cod_Listas_Precios = 1
    ORDER BY precio DESC);

--Clientes de "Zona Sur" que nunca realizaron un pedido (IN / NOT IN)
--Queremos detectar oportunidades comerciales buscando qué distribuidores de la "Zona Sur"
--todavía no han registrado ninguna compra en nuestra tabla de pedidos.

SELECT CUIT, RazonSocial, email
FROM Distribuidores
WHERE ZonaCobertura = 'Zona Sur'
AND CUIT NOT IN (
    -- Esta subconsulta devuelve una lista de una columna con todos los CUITs que ya compraron
    SELECT DISTINCT CUIT_Cliente
    FROM Pedidos_Distribuidores );
```

- **Cuidado con los NULLs en NOT IN:** Si la subconsulta devolviera un valor NULL, la consulta completa con NOT IN fallará y no devolverá ningún registro. Para evitarlo, siempre agrega WHERE columna IS NOT NULL dentro de la subconsulta.
- **Operadores correctos:** Si la subconsulta devuelve más de un valor, no puedes usar =, < o >. Debes usar operadores de conjunto como IN o acompañarlos de ANY / ALL.

EXISTS - El comando EXISTS (Evaluación de Existencia)

EXISTS no se fija en los valores que devuelve la subconsulta; solo le importa si esa subconsulta **devuelve al menos una fila** o ninguna. Es binario: si encuentra una coincidencia, es TRUE.

- **¿Cuándo se usa?** Cuando necesito validar relaciones de tipo *"tiene o no tiene"*.
- **Ejemplo:** Queremos listar los nombres de todos los **Empleados** que **tienen hijos** registrados.

```
SELECT e.num_legajo, p.nombre, p.apellido
FROM Empleados e
INNER JOIN Personas p ON e.DNI = p.DNI
WHERE EXISTS (
    SELECT 1
    FROM Hijo_Empleado h
    WHERE h.DNI_Empleado = e.DNI);
```

```
✓ SELECT e.num_legajo, p.nombre, p.apellido
   FROM Empleados e
   INNER JOIN Personas p ON e.DNI = p.DNI
   WHERE EXISTS (
       SELECT 1
       FROM Hijo_Empleado h
       WHERE h.DNI_Empleado = e.DNI);
```

ANY - El comando ANY (Comparación contra una Lista)

ANY requiere que pongas un operador de comparación antes (por ejemplo, > ANY o = ANY). Evalúa tu valor contra **cada uno** de los elementos que devuelve la subconsulta. Si la condición se cumple para **al menos uno** de los elementos de la lista, devuelve TRUE.

- **¿Cuándo se usa?** Cuando necesito comparar magnitudes (fechas, montos, cantidades) contra un grupo de registros.
- **Ejemplo:** Queremos buscar qué **Artículos** de nuestra base de datos tienen, en **alguna** de las listas de precios, un valor superior a los \$4500 (pesos).

```
SELECT ArtículoID, Nombre
```

```

FROM articulos

WHERE ArtículoID = ANY (

    SELECT ArtículoID
    FROM articulos_Precios
    WHERE precio > 4500);

```

```

SELECT ArtículoID, Nombre
FROM articulos
WHERE ArtículoID = ANY (
    SELECT ArtículoID
    FROM articulos_Precios
    WHERE precio > 4500);

```

Cómo funciona: La subconsulta devuelve una lista de IDs de artículos caros (ej: [5, 12, 24]). Luego, la consulta principal evalúa cada artículo: *¿Mi ID es igual a alguno de esa lista?* Si coincide con cualquiera de ellos, pasa el filtro.

Nota técnica: = ANY funciona exactamente igual que el operador IN. Si usaras > ANY (lista_de_precios), buscaría valores que sean mayores que el precio más bajo de esa lista.

Tabla comparativa final de comportamiento

Escenario	Con EXISTS	Con ANY
Rendimiento	Más rápido si la subconsulta maneja muchos datos (frena al primer acierto).	Puede ser más lento si la lista devuelta es extremadamente gigante.
Sintaxis	Siempre se escribe WHERE [NOT] EXISTS (subconsulta).	Requiere un operador previo: WHERE columna [=, >, <] ANY (subconsulta).
Nulos (NULLs)	Ignora si la subconsulta devuelve valores NULL (solo busca filas).	Si la subconsulta devuelve un NULL, la comparación completa puede dar UNKNOWN.

SELECT: Las subconsultas en la cláusula SELECT se conocen como **subconsultas escalares**. Se ejecutan por cada fila que devuelve la consulta principal y tienen una regla de oro inquebrantable: **deben devolver exactamente un único valor**

(una fila y una columna). Si devuelven más de un dato, el motor SQL arrojará un error de ejecución inmediato.

Son herramientas excelentes para añadir columnas de cálculo, totales agrupados o datos específicos de auditoría directamente en el reporte principal, sin necesidad de alterar los GROUP BY globales.

Ventajas y Desventajas de las subconsultas en el SELECT

- **Ventaja principal:** Te permiten traer datos de tablas relacionadas **sin duplicar las filas** de la tabla principal (un problema común cuando abusamos de los LEFT JOIN con tablas de detalle) y te ahorran escribir agrupaciones complejas en la consulta externa.
- **Desventaja de rendimiento:** Se ejecutan **fila por fila** (operación tipo *RBAR: Row-By-Agonizing-Row*). Si tu consulta principal devuelve 1.000.000 de filas, la subconsulta del SELECT se va a ejecutar un millón de veces, lo que puede destruir el rendimiento de tu servidor.

```
--Reporte de depósitos con contador de ubicaciones. Queremos listar los datos de los depósitos
--y, en una columna adicional, mostrar cuántas ubicaciones físicas tiene configuradas cada uno de ellos.

SELECT
    d.Cod_deposito,
    d.descripcion,
    d.capacidad_total_m3,
    (SELECT COUNT(*)
     FROM Ubicaciones u
     WHERE u.Cod_deposito = d.Cod_deposito) AS Cantidad_Ubicaciones
FROM Depositos d;

--Listado de artículos con su precio de lista y desvío. Queremos armar un catálogo de artículos
--que muestre su nombre, su precio en la Lista General Minorista (Lista 1)
--y su precio en la Lista de Distribuidores (Lista 2).

SELECT
    a.Nombre,
    (SELECT ap.precio
     FROM articulos_Precios ap
     WHERE ap.ArticuloID = a.ArticuloID AND ap.Cod_Listas_Precios = 1) AS Precio_Minorista,
    (SELECT ap.precio
     FROM articulos_Precios ap
     WHERE ap.ArticuloID = a.ArticuloID AND ap.Cod_Listas_Precios = 2) AS Precio_Distribuidor
FROM articulos a;
```

USO DE REFERENCIAS EN SUBCONSULTAS

Las subconsultas que utilizan **referencias externas** se conocen técnicamente como **subconsultas correlacionadas**.

A diferencia de una subconsulta común (que se ejecuta una sola vez y le pasa el resultado a la consulta principal), una subconsulta correlacionada **se ejecuta fila por fila**. El motor toma un dato de la consulta externa (la referencia), lo introduce en la consulta interna, procesa el resultado y pasa a la siguiente fila.

```

--Última orden de compra realizada a cada proveedor. Queremos un listado de todos nuestros proveedores
--y necesitamos saber la fecha exacta en la que les emitimos la última Orden de Compra.
SELECT
    p.Id_interno_Prov,
    p.nombre_empresa,
    (SELECT MAX(c.fecha_alta)
     FROM Compras c
     WHERE c.Id_interno_Prov = p.Id_interno_Prov) AS Fecha_Ultima_Compra
FROM Proveedores p;

```

```

--Queremos un listado de Artículos. Para cada uno, queremos ver su nombre y,
--en una columna adicional, el precio más alto al que ha sido comprado
--en la historia de las Órdenes de Compra
SELECT
    a.ArticuloID,
    a.Nombre,
    -- Subconsulta en el SELECT con referencia externa (ic.ArticuloID = a.ArticuloID)
    (SELECT MAX(ic.precio)
     FROM Items_Compra ic
     WHERE ic.ArticuloID = a.ArticuloID) AS Precio_Maximo_Comprado
FROM articulos a
order by 3 asc

```

```

----Queremos encontrar a los Empleados que ingresaron a la empresa
--antes que el empleado con el legajo número 1005
--(queremos ver quiénes tienen más antigüedad que él en toda la organización).

```

```

SELECT
    e_externo.num_legajo,
    p.nombre,
    p.apellido,
    e_externo.fecha_ingreso
FROM Empleados e_externo
INNER JOIN Personas p ON e_externo.DNI = p.DNI
WHERE e_externo.fecha_ingreso < (
    -- Esta subconsulta busca la fecha exacta de un empleado específico
    SELECT e_interno.fecha_ingreso
    FROM Empleados e_interno
    WHERE e_interno.num_legajo = 1005);

```